



南京工业大学
NANJING TECH
UNIVERSITY

数据挖掘与机器学习

信管2301

黄陈熙

202321054011





南京工业大学
NANJING TECH
UNIVERSITY

第一讲 数据挖掘的基本认知

信管2301

黄陈熙

202321054011



课后作业

阅读《白话机器学习算法》第1章，整理读书笔记。

一、章节核心定位

- 1.主题定位：阐明“大数据挖掘”的基本概念、数据挖掘与大数据的关系、数据挖掘的内容与应用场景，以及入门级的工具与流程。
- 2.目标读者与应用导向：面向本科/研究生层级的读者，强调理论结合实践，提供MATLAB等工具的实际应用示例，帮助理解数据挖掘的全生命周期。

课后作业

二、第1章的核心要点（按书中段落与要点梳理）

1.1 大数据与数据挖掘

大数据的核心特征：体量大（Volume）、数据多样性（Variety）、价值密度低（Low Value Density）、速度快（Velocity），简称常用的四个V概念。

大数据的价值：不仅在于数据量的放大，更在于通过挖掘从海量数据中发现有价值的模式和规律，从而支持决策与业务优化。

关系与区分：大数据是数据挖掘的一种外延表达，数据挖掘是在大数据背景下对数据进行知识发现的系统方法。

应用驱动的核心观点：大数据的最终价值来自于实际应用场景的落地与商业化价值，而非单纯的数据堆积。

1.1.1 什么是大数据

通过必胜客的CRM场景等故事阐明数据贯穿业务流程，强调数据的获取、存储、集成与使用的链条。

数据集成与统一管理的重要性：把分散数据整合，有利于后续的挖掘与分析。

降维与降维策略的初步提及：对海量数据在计算与分析中的可行性考虑，以及分层/逐步降维的思路。

1.1.2 大数据的价值

通过故事情节引出对价值的理解：数据在经过分析、学习后，能揭示隐含规律，指导未来决策。

大数据价值的持续性：价值不会因为使用而减少，可以在不同场景多次复用。

课后作业

1.2 数据挖掘的原理

数据挖掘是多学科的综合应用：数据库、统计学、应用数学、机器学习、可视化、程序开发等共同构成数据挖掘的基础。

数据挖掘的流程与输出：从输入数据到输出模型，经过训练、验证，最终用于预测与解释。

数据挖掘的核心目标：建立具有泛化能力的模型，能够对未知数据进行准确预测。

1.3 数据挖掘的内容

主要内容六大类核心方法：关联规则、回归、分类、聚类、预测、诊断（后文章中会展开每一类方法及对应算法）。

数据挖掘前期技术：数据准备（收集、质量分析、预处理）与数据探索（衍生变量、降维、可视化等）。

时序数据挖掘与智能优化：时间序列方法、遗传算法、模拟退火等。

数据挖掘过程的人文与流程性要素：商业目标设定、数据理解、数据质量、数据预处理、数据探索等环节的协同。

1.4 数据挖掘的应用领域与未来趋势（章节摘要）

数据挖掘在零售、银行、证券、能源、医疗、通信、汽车等行业的潜在应用场景。

对大数据时代的思考：要避免“技术驱动”而忽视应用，强调“数据驱动的应用”与行业知识的重要性。

对数据隐私、伦理、成本与平台建设的理性认知。

课后作业

1.5 大数据挖掘的要点（要点小结）

大数据思维：即便数据量大，也应关注能否从数据中提取有用信息与知识，强调开放、共享、协作的态度。

数据平台成本与架构的意识：大数据平台建设的成本、能耗、维护等要素需要在项目初期就被权衡。

数据挖掘的“艺术”与“科学”的结合：不仅要掌握算法，还要理解业务目标、数据特征及实际约束。

强调“人机协作”在数据挖掘中的角色：工具再强，人的商业判断、领域知识和决策能力仍然关键。

1.6 本章的阅读路径与学习建议

先建立对大数据与数据挖掘关系的宏观认知，再进入数据挖掘的流程与前期技术（数据准备、探索）。

逐步理解六大核心挖掘方法的基本原理与应用场景，结合书内 MATLAB 实例进行实践。

注重案例学习，将理论与实际数据结合，体会从数据准备到模型部署的完整过程。

课后作业

三、对学习的思路与笔记提炼要点

学习路线建议

阶段1：理解大数据与数据挖掘的关系、核心概念与价值定位。

阶段2：熟悉数据挖掘的流程（CRISP-DM/SEMMA等常见模型），掌握数据准备与数据探索的要点。

阶段3：初步掌握六大核心方法的基本思想与适用场景，关注算法的优缺点与适用条件。

阶段4：了解智能优化与时序数据挖掘的基本思路，初步接触 MATLAB 实现与工具（如分类学习机、PCA 等）。

阶段5：结合行业应用案例，分析业务目标、数据源、指标、评估方法等要素。

重要概念与术语

数据挖掘过程模型：CRISP-DM（Business Understanding、Data Understanding、Data Preparation、Modeling、Evaluation、Deployment）。

数据准备要素：数据收集、数据质量分析、数据预处理（清洗、集成、归约、变换）。

核心方法类别：关联规则、回归、分类、聚类、预测、诊断。

降维与变量处理：主成分分析（PCA）、衍生变量、离散化、规范化。

实践与工具

书中以 MATLAB 为工具，提供了“MATLAB 数据挖掘快速入门”和“MATLAB 集成数据挖掘工具 Classification Learner”等内容的实操导向。

关注代码实现与案例应用，理解从数据读取、预处理、建模到评估的完整流程。

课后作业

四、可操作的学习清单

阅读与笔记

梳理大数据四要素与价值的关系，摘录定义与场景。

列出数据挖掘六大核心方法的基本原理与常用算法，并标注适用场景。

记录CRISP-DM六大阶段的要点与具体活动。

实践部分

运行书中提供的 MATLAB 快速入门实例，完成数据读取、探索、预处理、建模的最小可行流程。

使用 Classification Learner 对一个小数据集进行分类任务，比较不同分类器的性能。

结合具体行业案例，尝试给出一个简短的数据挖掘计划草案（目标、数据源、评估指标、潜在风险）。

思考与应用

反思“技术驱动”与“应用驱动”的关系，写一段短评说明在实际项目中如何平衡两者。

思考数据质量、数据隐私与成本在大数据项目中的权衡点，并列出可执行的改进点。

课后作业

整理本章上页PPT“本章概念汇总”，形成笔记。

1. 大数据（Big Data）

理解要点（教材侧重点）：大数据的核心特征不只是“数据大”，而是与之相关的复杂性与价值获取方式。教材在“何为大数据/大数据特点”相关章节强调大数据具有体量大、多样性、价值密度低、速度快等特点（用于说明大数据分析与传统分析的差异）。

2. 数据挖掘（Data Mining）

教材给出了较典型的定义方向：

数据挖掘（DataMining）：从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，提取隐含的、事先不知道的、有价值的信息/知识。

教材还进一步指出：数据挖掘的目标是发现隐藏规律，并在一定程度上自动化这一过程（相对传统 OLAP 的更手工/更偏分析报告的方式）。

课后作业

3. 数据降维（Dimensionality Reduction）

教材在“大数据的降维”中给出方法路线：

大数据量可能超过处理能力，因此处理后通常需要降维；

降维可以分级逐层做：

1) 抽样降维

2) 抽取有用变量

3) 再用经典降维方法（如 PCA）做进一步的“变形降维”（把高维映射/变换到低维）

教材也提到另一类思想：

分散式降维：将大数据映射为小单元计算，再对结果整合（与 Map-Reduce 框架思路相关）。

4. 有监督（supervised） / 无监督（unsupervised）

教材在“数据挖掘模型结构/流程”里给出了一个很实用的归纳：

数据挖掘常见的模型可分为 有监督模型 和 无监督模型。

有监督：训练数据中有“已知结果/标签”，模型学习输入到输出的映射（例如：分类、回归、预测等常对应有监督框架）。

无监督：训练时没有标签，主要做结构发现，例如 聚类、关联 等无监督任务。

教材在图示中列举了示例算法方向（例如：PCA 放在“数据降维”常联系的无监督结构里；聚类 K-means 等；有监督里回归/分类/预测等）。

课后作业

5. 结构化 / 半结构化 / 非结构化数据

教材在“数据的存在形式”里给了非常清晰的划分与例子：

结构化数据（Structured Data）：简单说就是数据库；典型场景：ERP、财务系统、医疗HIS、教育一卡通、政府审批等。

非结构化数据（Non-Structured Data）：视频、音频、图片、图像、文档、文本等；典型场景：医疗影像、教育视频点播、视频监控、GIS、文件服务器等。

半结构化数据（semi-Structured Data）：邮件、HTML、报表、资源库等；典型场景：邮件系统、WEB集群、教学资源库、档案系统等。

并且教材还把三者对应的系统需求也顺带给了：存储、备份、共享/归档等。

6. 数据预处理（Data Preprocessing）与数据清洗（Data Cleaning）

教材把数据准备/预处理作为数据挖掘的重点环节，并指出：

数据几乎不可能完美，现实世界会有缺失、噪声、错误、不一致等问题；为得到准确、完整、一致性较好的数据，必须做预处理。

课后作业

7. 数据标准化（Normalization）与数据归一化（你这里可能需要区分概念）

教材在“数据变换—标准化”部分给出了标准化（Normalization）的定义与常用方法：

标准化（Normalization）：把数据按比例缩放，使其落入一个小的特定区间，目的是去除单位限制/让不同量级指标便于比较与加权。

教材特别点了两类常用：

0-1 标准化（离差标准化）：映射到 $[0,1]$

Z-score 标准化（标准差标准化）：映射到标准正态（均值 0，标准差 1）

8. 衍生变量（Derived Variable）

教材在“衍生变量”章节给出定义：

衍生变量：由已有变量通过不同形式的组合/变换得到的新变量。

例如：已知质量、长度、体积，可以衍生密度 = 质量/体积、线密度 = 质量/长度

教材还强调在数据挖掘过程中通常需要衍生变量以获得更多可用信息。

数据回归分析与模型探究

线性回归、Logistic回归及《白话机器学习》读书笔记

Data Regression Analysis

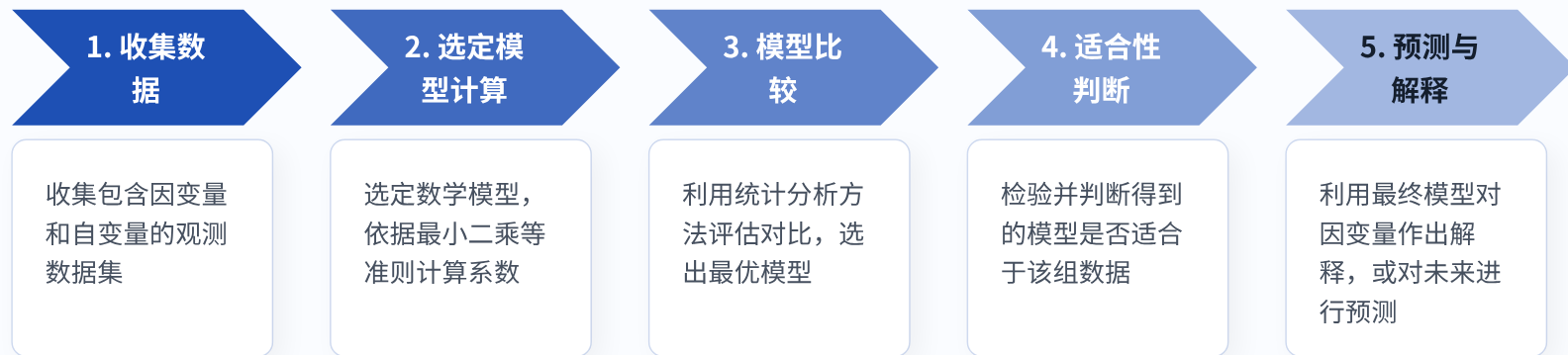


汇报目录

AGENDA

1. 数据回归方法与基本步骤
2. 一元与多元线性回归模型
3. Logistic回归模型及其特点
4. 线性与Logistic回归的区别与联系
5. 《白话机器学习》第六章读书笔记

建立回归模型的5个标准步骤



一元线性回归探究单一变量的线性影响

1. 模型含义

一元线性回归是最基础的数据回归模型，主要用于探究单一自变量对因变量产生的线性影响。

2. 变量释义

- Y ：因变量
- x ：自变量（非随机变量因素）
- β_0, β_1 ：未知的回归系数
- ε ：随机误差，满足均值 $E(\varepsilon) = 0$ ，方差 $\text{var}(\varepsilon) = \sigma^2$

基本数学模型

$$Y = \beta_0 + \beta_1 x + \varepsilon$$

经验模型（利用最小二乘法估计）

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

模型评估：决定系数

$$R^2 = 1 - \frac{SSE}{SST}$$

R^2 越接近 1，表示观测值与拟合值越靠近，回归效果越好。

多元回归引入多个变量以衡量综合影响

基本数学模型

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$$

矩阵表达形式

$$Y = X\beta + \varepsilon$$

矩阵构成

- $Y = [y_1, y_2, \dots, y_n]^T$
- $\beta = [\beta_0, \beta_1, \dots, \beta_p]^T$
- $\varepsilon = [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n]^T$

多因素综合分析

现实世界中，结果往往受多个因素共同作用。多元线性回归通过引入 p 个非随机解释变量，大幅提升了对复杂系统的刻画能力。

核心变量释义

- **观测向量** Y ：被解释变量组成的 n 维列向量
- **设计矩阵** X ：自变量观测值组成的数据矩阵
- **待估计向量** β ：包含全部回归系数
- **随机误差** ε ：假定 $\varepsilon \sim N(0, \sigma^2 I_n)$

Logistic回归通过指数映射解决分类预测

当因变量是**定性变量**（如 0 或 1，代表企业破产或健康、股票涨跌）时，普通线性回归不再适用。此时采用 Logistic 回归，以探究某分类现象发生的**概率 p** 与各因素的关系。

Logistic 回归基本形式（概率预测）

$$p(Y = 1|x_1, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

对数变换机制

对上述公式进行变形，即可将其转化为线性回归结构：

$$\ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

实际映射处理

由于样本中 Y 只能取 0 或 1，直接求对数无意义。实际中常定义一种单调连续的概率函数 π ($0 < \pi < 1$)，将原始数据进行映射处理，最后反映射求出真实 p 值。

线性回归与Logistic回归的本质差异

普通线性回归 (Linear Regression)

预测目标

预测**定量的连续型变量**。例如：预测具体的房屋价格、下个季度的销售额、身高等。

数学联系

直接拟合自变量与因变量的线性关系方程式：

$$Y = X\beta + \epsilon$$

适用场景

适用于各变量间具有明确的线性相关性场景，用以解释系统变化规律。

逻辑回归 (Logistic Regression)

预测目标

预测**定性的离散型变量**。本质上是计算某一分类事件发生的**概率大小**，而非绝对数值。

数学联系

通过对数变换，巧妙地将概率 p 转化为线性回归问题求解。
两者在**参数估计上具有相似的数学处理逻辑**。

适用场景

广泛应用于疾病诊断（是否患病）、信用风险评估（是否违约）等“是与非”二分类预测任务。

《白话机器学习》笔记 (1): 趋势与优化思路

1. 寻找最佳趋势线

回归分析的核心在于寻找最佳拟合线（趋势线）。趋势线不仅是最直观的预测工具，还在本质上反映了数据演变的方向。

通过构建带权重的**组合预测变量**：

- 能够极大改善预测结果的准确度
- 使得直接比较各个预测变量的“强弱”影响成为可能，增强模型的可解释性

2. 梯度下降的优化哲学

当无法通过直接解方程获取最优权重时，梯度下降法提供了极为关键的通用求解思路。

算法运作机制：

它首先初步猜测一组权重，接着通过不断迭代调整权重——永远朝着使整体预测误差下降“最陡”的方向前进，最终抵达误差最低点。

*注：普通回归分析中极少陷入局部最优（凹坑），该方法稳健性较强。

《白话机器学习》笔记 (2): 系数认知与局限性

3. 标准化回归系数

回归系数代表变量的**相对增量影响**。但若度量单位不同，直接比较系数大小会导致严重误判。

标准化处理至关重要：

只有经过单位统一（标准化）后的“标准化回归系数”，才能准确比较不同变量的强弱。在单变量模型中，它直接等同于**相关系数**，反映了关联方向与强度。

4. 模型的局限性反思

在实际应用中，数据回归存在四项不可忽视的天然局限：

- **对异常值极度敏感：** 少数离群点可能拉偏整条趋势线
- **多重共线性问题：** 高度相关的预测变量组会导致各自的权重严重失真
- **仅限线性趋势：** 无法天然拟合复杂的弯曲演变
- **因果谬误：** 回归只揭示强“相关”，绝不代表两者间存在“因果”

感谢聆听

数据回归为理解世界提供了量化的视角

Classification & KNN

机器学习分类算法与KNN原理

核心逻辑探究及《白话机器学习》读书笔记

汇报目录

AGENDA

1. 分类算法的基本概念
2. 分类算法的训练测试逻辑
3. KNN算法的核心思想
4. KNN算法的具体步骤

5. 读书笔记：KNN的调参与局限
6. 读书笔记：决策树的生成原理
7. 读书笔记：决策树的优劣与演进

分类算法的核心概念框架

分类 (Classification)

对现有数据进行学习，得到一个目标函数或规则，把每个属性集准确映射到一个预先定义的**类标号**。

训练集 (Training Set)

由已知特定类标签的数据记录组成。每一条记录包含若干属性构成的特征向量。它是系统生成模型的**经验输入**。

模型 (Classification Model)

通过学习得到的目标函数或规则。

- **描述性建模**：用于区分不同类对象的解释性工具。
- **预测性建模**：用于预测未知记录类别。

分类算法“两步走”的基础逻辑

在构造模型之前，首先需将带类标号的完整数据集随机划分为**训练数据集**和**测试数据集**。

第一阶段：训练 (归纳)

有指导的学习：

系统分析训练集中各属性描述的数据元组，归纳其与预定义类标号间的内在联系，从而建立能够准确拟合数据的**分类模型**。

第二阶段：测试 (评估)

检验泛化能力：

使用未参与训练的检验集 (Test Set) 对已生成的模型进行评估。测算该模型对未知记录的**分类准确率**。

最终应用：推论预测

投入真实业务场景：

若模型在测试阶段的准确率可接受，则将其应用于业务，对未来带有未知类标号的新输入数据进行正式的**预测判定**。

KNN算法基于邻近度的核心判别思想

1. 核心理念

“物以类聚，人以群分”。给定一个类标号未知的样例，找出距离它最近的 **K 个数据点**（即 K-最近邻）作为判定参考。其依据在于相同特征空间内的点往往具有相似的属性。

2. 判别准则

采取**多数表决方案**：如果这 K 个近邻中包含多个类标号，则将该未知样例指派到其最近邻中出现频率最高（多数）的类。若平局则可随机选择。

算法应用拓展

除了用于离散类别的**分类判定**外，KNN 也可用于**连续值预测**：

不再使用简单的多数表决，而是通过合计周围 K 个数据点的值进行**加权平均**（距离越近的邻居权重越大），从而预估目标数据点的真实连续数值。

KNN算法的距离计算与迭代步骤

Step 1

初始化配置：初始化距离变量为系统最大值，准备接收后续计算。

Step 2

距离计算：逐个计算未知样本与训练集中每个已知样本的距离 (dist)。

Step 3

维护近邻集：得到目前K个最近邻样本中的最大距离 maxdist。如果 dist 小于 maxdist，则将该训练样本作为新的 K-最近邻样本更新集合。

Step 4

循环迭代：重复上述计算与比较步骤，直到未知样本和所有训练样本的距离都计算完毕。

Step 5

多数表决：统计此时确定的 K 个最近邻样本中每个类别出现的次数。最终选择出现频率最大的类别作为未知样本的类别。

KNN调优权衡与适用维度的局限性

关键参数：K 值的选择

KNN 算法的性能高度依赖周围参考邻居的数量（K 值）：

- **K值太小**：预测目标极易产生变动性，随机噪声所产生的误差会被放大。
- **K值太大**：模型尝试与更远的“邻居”匹配，决策面过于平滑，导致数据中隐含的真实模式被忽略。

在实践中，最佳的 K 值（二分类常设为奇数以防平局）通常需要通过**交叉验证法**来确定，以期将预测误差降至最低。

算法的两大核心局限

1. 类别不平衡问题：

当各类样本规模悬殊时，属于最小类别的少数派数据极易被占据多数的大类数据所“掩盖”。

解决对策：引入加权投票法，距离越近权重越大。

2. 预测变量过多（维度灾难）：

高维空间下距离计算量剧增，且距离度量可能失效；同时，大量无用冗余变量会干扰判定。

解决对策：预先采用降维技术提取最具影响力的特征。

决策树通过递归拆分追求最大同质性

1. 生成逻辑：递归拆分

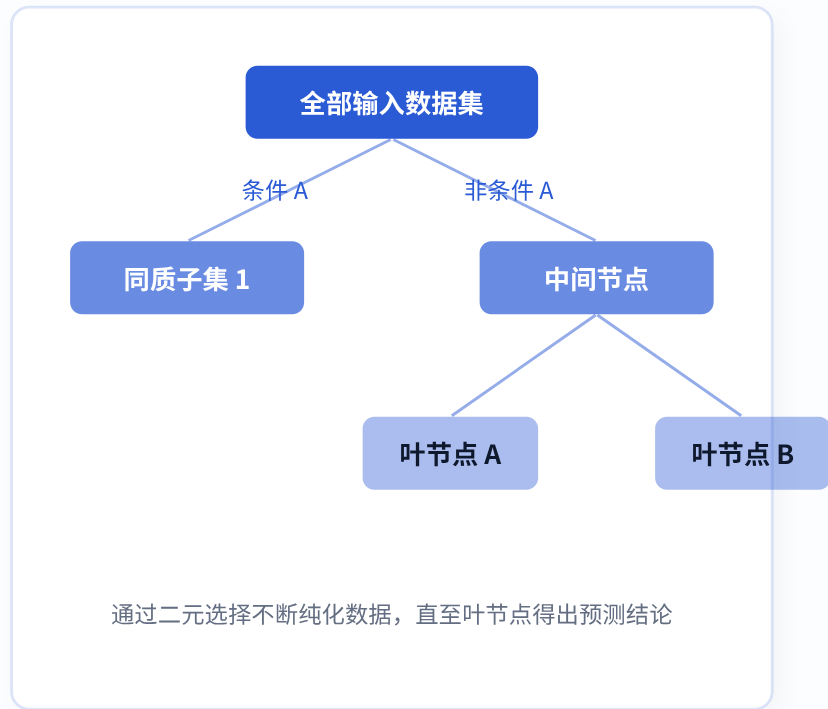
决策树通过询问一系列**二元选择题**（如：是否属于某一特征）不断向下分支。核心目标是把数据点进行拆分，并最大限度地提高每一次拆分后子群组的**同质性**（即同类数据聚集在一起）。

2. 具体生长步骤

- **步骤 1：** 确定一个最佳二元问题，将当前数据点一分为二。
- **步骤 2：** 针对产生的新节点重复步骤 1，以此类推。

3. 终止生长的条件

- 叶节点内的数据已全属同一类或完全同质；
- 节点内剩余的样本数量过少（如少于 5 个）；
- 进一步的分支已无法再提高群组同质性。



决策树的白盒优势与集成演进方向

模型的原生优势

- **解释性极强：**决策过程完全白盒化，极易实现可视化，非技术人员也能轻松理解树的判断逻辑。
- **耐扰性较高：**分支建立在最佳二元选择题之上，它能自动忽略那些不显著的变量，因而对少数异常值（离群点）有较好的天然抵抗力。

存在的核心痛点

- **极度不稳定：**数据样本的细微变动可能会让树的生长结构面目全非；为追求每个节点的最佳拆分，模型极易陷入“**过拟合**”陷阱。
- **贪心导致的次优解：**生成时遵循局部最优（贪心算法），每次都抓取当前最佳切割点，但这并不能保证整体生成的树具有全局最高准确率。

进阶演进：放弃单打独斗，拥抱集成模型

为了克服单棵树的不稳定与不准确，现代算法通过综合生成大量具有差异化的决策树组建“**随机森林**”，或采用“**梯度提升**”策略不断修正预测错误，从而在牺牲部分可视化的前提下，大幅增强模型的全局精度。

感谢聆听

掌握分类底层逻辑，构建稳健数据认知

—

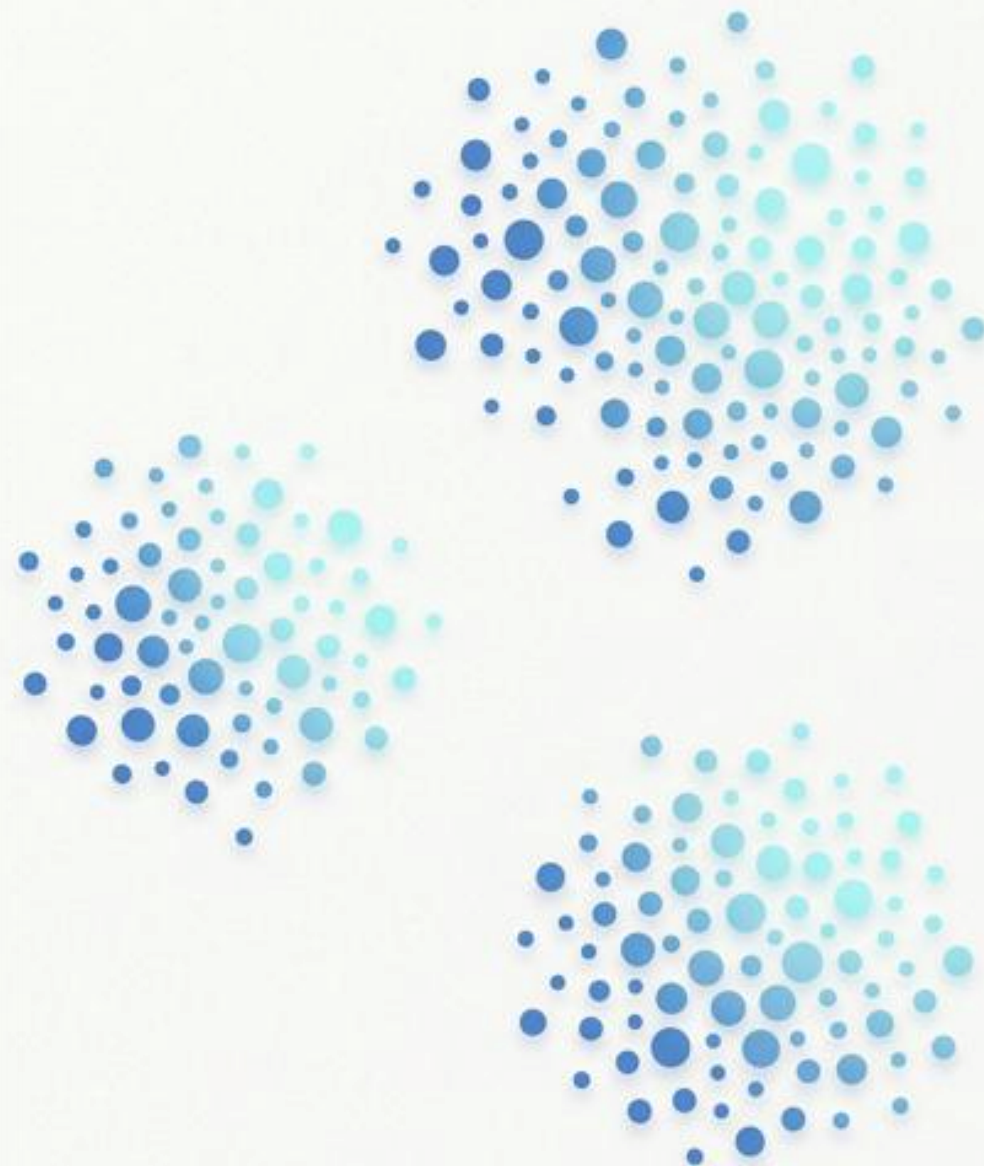
聚类分析

与K-means算法

无监督学习中的代表性方法

姓名：黄陈熙

学号：202321054011



目录

AGENDA

01 聚类与“簇”的概念

02 度量方法与距离

03 K-means原理与步骤

04 HC层次聚类流程

05 树状图与聚类解释

06 K-means读书笔记

07 K-means与HC对比

08 总结与Q&A

聚类与“簇”的概念

无监督条件下的数据探索与模式发现

01. 核心定义

聚类：在无监督条件下将数据划分为若干组，使组内相似度高、组间差异大。

簇：由相似样本构成的集合。

02. 为什么需要聚类

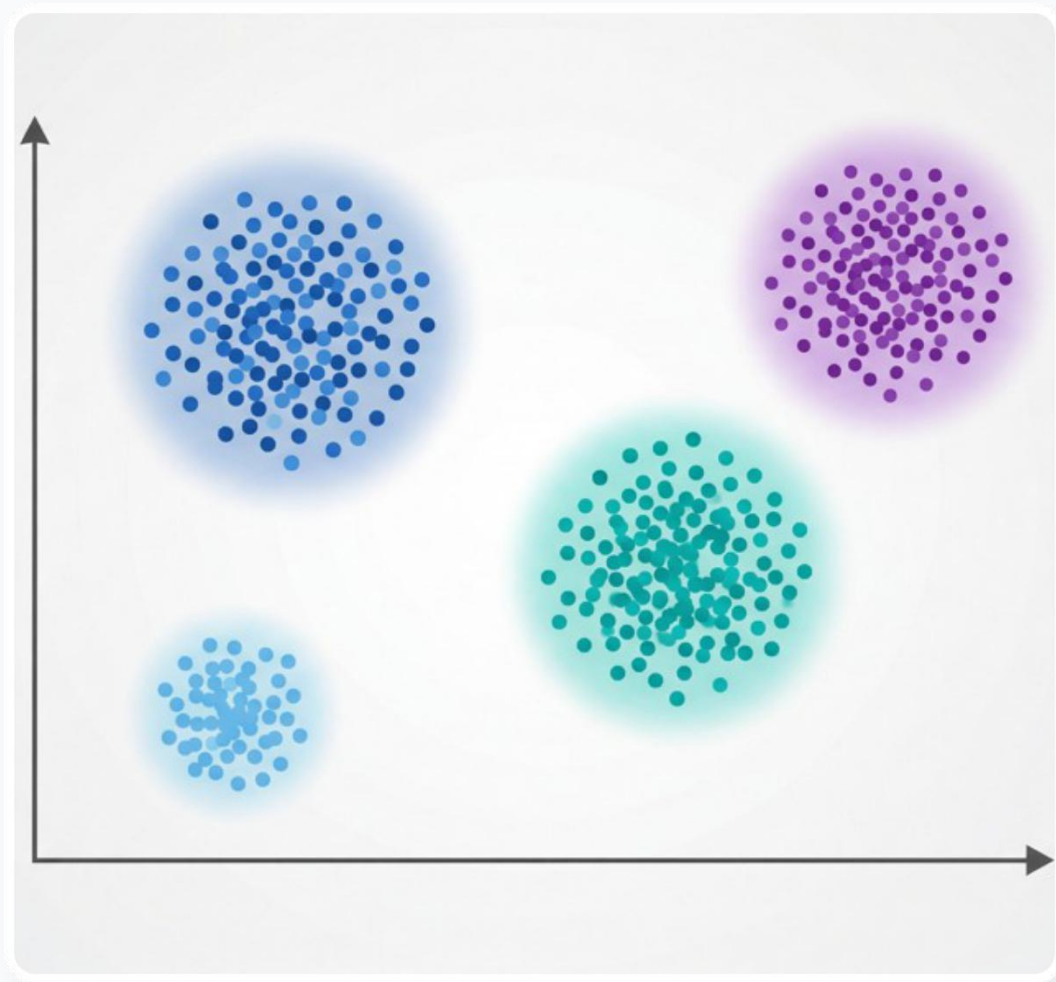
探索性分析：发现数据内在结构与模式（如用户分群、图像分割）。

表示学习：用簇作为代表点，实现数据压缩与降维。

03. 常见假设与优化目标

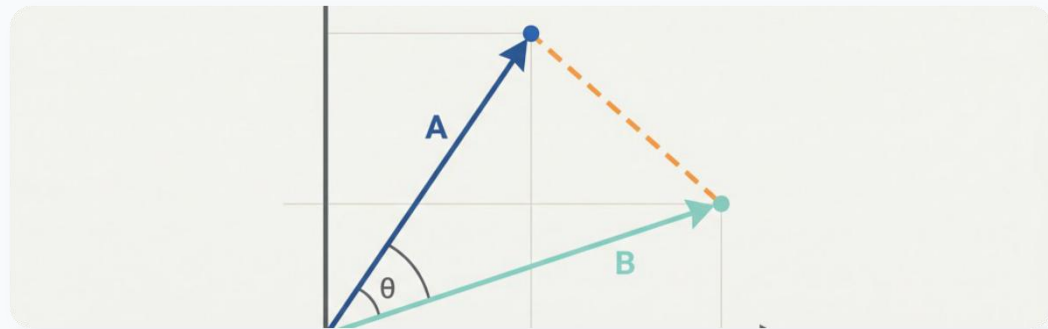
前提是样本间的**相似度可被定量度量**。

目标通常是使特定的目标函数（如簇内平方和）最小化，或形成最清晰的层次。



度量方法与距离

核心：如何定量衡量样本之间的“相似性”



欧式距离 (Euclidean Distance)

公式：空间中两点的直线距离。

直觉上就是几何距离。

特点：对数值“尺度”极敏感。

使用前必须进行标准化或归一化处理。

适用场景：连续数值型特征。

✓ 传感器数据、房价、尺寸预测等具备物理量纲的数值。

VS

余弦距离 (Cosine Distance)

公式：衡量两向量夹角，距离 = $1 - \cos(\theta)$

计算向量间的方向一致性。

特点：关注“方向”而非“长度/大小”。

对文本等特征的绝对频次不敏感。

适用场景：高维稀疏特征。

✓ 文本挖掘(TF-IDF)、词向量(Word Embedding)、推荐系统偏好。

K-means: 原理与步骤

最小化簇内平方和(SSE)的交替优化策略

初始化质心

选择簇数 K ，并随机或使用K-means++策略初始化 K 个质心。

分配样本 (Assignment)

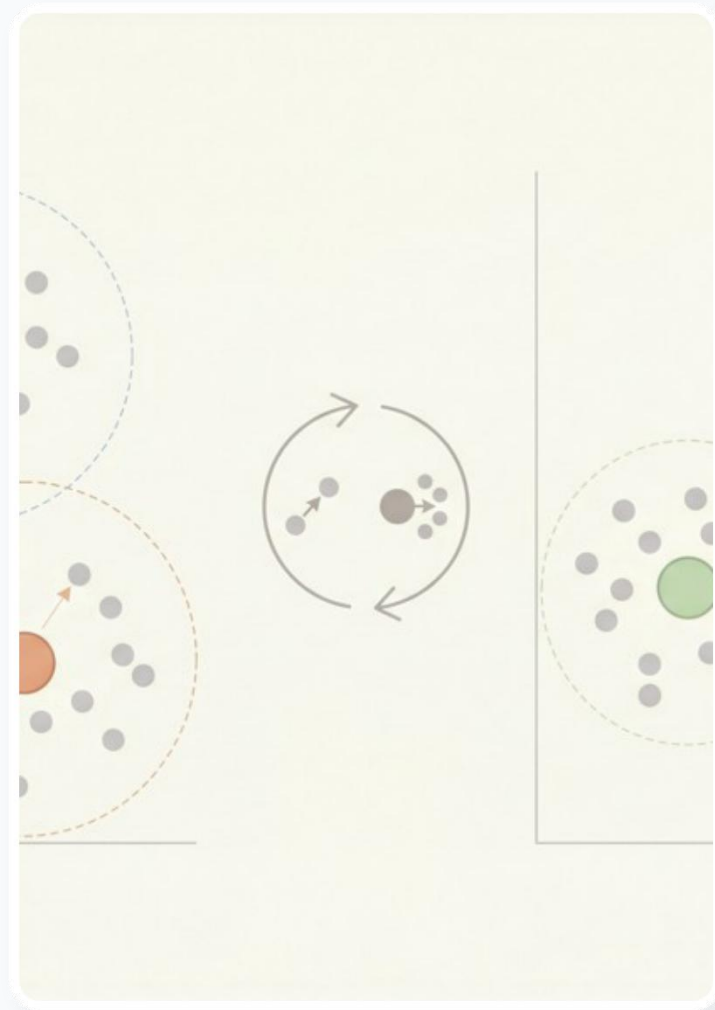
计算每个样本到所有质心的距离，将其指派给距离最近的簇。

更新质心 (Update)

固定簇划分，用当前簇内所有样本的均值向量更新该质心位置。

判断收敛

若质心位置几乎不再变化，或达到最大迭代次数上限，则算法停止；否则返回步骤2。



HC：凝聚型层次聚类算法过程

自底向上构建层次化簇结构的过程



簇间距离 (Linkage) 选择策略

Single Linkage (最短距离)

两簇中最接近样本间的距离。易引发链式反应。

Complete Linkage (最远距离)

两簇中最远样本间的距离。产生的簇较紧凑。

Average Linkage (平均距离)

两簇所有样本间距离的平均。折中方案稳健性好。

Ward法 (离差平方和) 最常用

最小化合并后的簇内方差增量，常与欧式配合。

树状图与聚类划分

如何看懂Dendrogram并选择切割线(Cut Height)

树状图怎么读？

纵轴：表示合并距离（或相异度），合并时的纵轴高度越高，说明这两部分差异越大。

横轴：底部节点代表单个样本或非常小的初始簇。

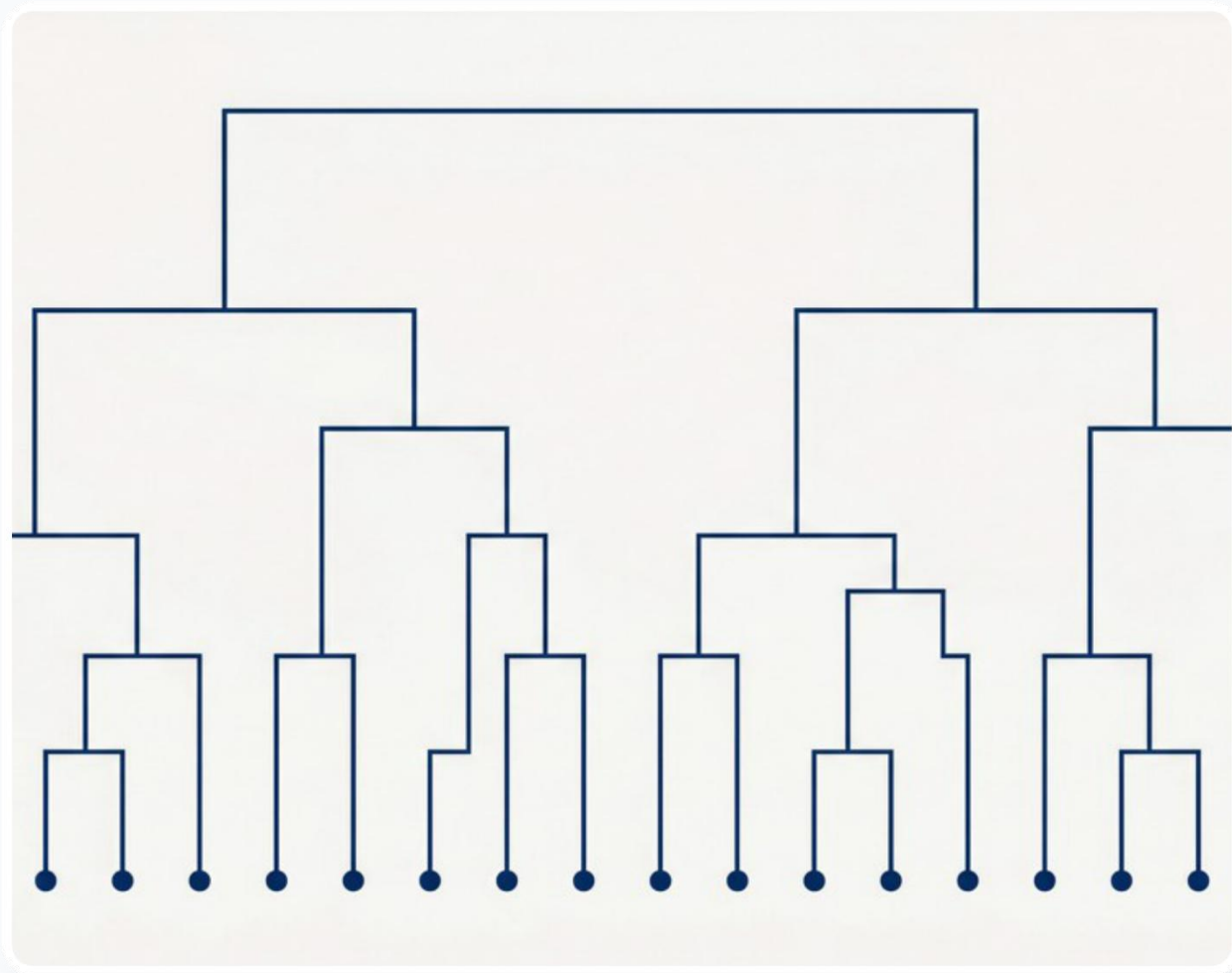
“聚成5类”的几何意义

在树状图上画一条水平的**切割线(Cut line)**。当这条水平线从上往下移动，直到与树状图的交叉点（向下的竖线分支）正好为 **5** 个时，得到的就是 5 个簇。

实践切割建议

寻找最大跳跃：在纵轴上合并距离出现“最长一段无合并的空白区”切割最稳健。

结合业务可解释性：指标最优不等于业务最优，需配合实际分析。



《白话机器学习》读书笔记

K-means章节核心要点摘录与个人总结

“K-means 本质上是‘用 K 个中心点代表整个数据集’的迭代分配-更新方法，简单高效但十分依赖合理的 K 值选择与初值设定。”

直觉与迭代

质心就是簇的“代表”。让每个点不断向代表靠拢，代表再根据身边的人重新找中心。分配→更新→重复。

易错：距离主导问题

特征间尺度不一致时，数值大的特征会“吃掉”数值小的特征（距离主导）。聚类前必须进行数据标准化处理。

初始化“看脸”

如果初值落在了错误位置，容易陷入局部最优解。实务中应多次随机运行取SSE最小，或使用K-means++。

何时会失败？

偏好“近似球形、大小相近”的簇。当面对非球形（如环状）、或者密度/大小相差极大的分布时容易失效。

K-means 与 层次聚类(HC) 核心对比

理解算法边界，在实际业务中做正确的选择

对比维度	K-means	Hierarchical Clustering (HC)
输入需求	需提前指定 K 值并给定初始点	无需预先给定 K，事后切割决定
输出形式	对数据集的“一次性固定硬划分”	全连接的层次树状图（Dendrogram）
时空复杂度	通常为 $O(N)$ 级别，对大数据较友好	通常为 $O(N^2) \sim O(N^3)$ 级别，耗内存算力
簇形状假设	偏好“近似球形、尺寸相近”的簇	对形状无绝对假设（取决于选用的 Linkage），Single 更容忍不规则形状

选型建议：

数据量大、需要快速产出迭代时首选 **K-means**；想探索内在细致结构、样本量中小且注重解释性时优先 **HC (搭配Ward法)**。

三句总结 Takeaways

1. 聚类是无监督学习的核心工具，关键在于“相似度度量”的选择。
2. K-means 用质心迭代优化，适合大规模快速分群，但依赖 K 与初始化。
3. HC 提供层次结构与树状图，便于解释，但计算成本更高。

事务矩阵中行与列的严谨定义

关联规则挖掘的基础是事务数据库（Transaction Database），它通常被抽象为一个布尔型矩阵。在这个矩阵中，行和列有着特定的业务含义。

行 (Row) 的定义

代表一个**独立的事务（Transaction）**，是分析的最小独立样本单位。

- **在商品关联分析中：**

一行代表一次顾客的购买行为，例如一张超市购物小票或一笔网络订单。

- **在关联选股分析中：**

一行代表一个特定的时间观察窗口，例如某一个具体的交易日或交易周。

列 (Column) 的定义

代表一个参与关联计算的**具体项目（Item）**。

- **在商品关联分析中：**

一列代表一种具体的商品品类，如“牛奶”、“面包”、“尿布”。

- **在关联选股分析中：**

一列代表一个具体的行业板块，或者单只股票。

注：矩阵内的元素值通常为1（项目在事务中出现/行业大涨）或0（未出现/未涨）。

Association Rules & Apriori

关联规则与Apriori 算法应用探究

关联选股、矩阵定义、商品图谱与算法流程



汇报目录

1. 行业关联选股的背景知识

2. 关联分析中矩阵行与列的定义

3. 商品关联分析：随机数据与知识图谱

4. Apriori算法的基本思想

5. Apriori算法的具体过程

利用行业联动滞后性进行关联选股

背景痛点

在复杂多变的股市中，精准选择个股难度极大。但由于资金轮动和产业链的上下游关系，行业板块之间往往存在明显的联动效应。

关联策略思路

- **挖掘强关联**：利用数据挖掘从历史交易数据中找出具有强“共涨”关系的行业组合。
- **捕捉滞后补涨**：当某一个或多个先导行业出现明显涨势时，若其强关联的跟随行业尚未启动，则在该跟随行业中选择典型代表股票买入。

先导行业 (启动)



跟随行业 (未动)



买入滞后行业的典型个股
等待补涨效应

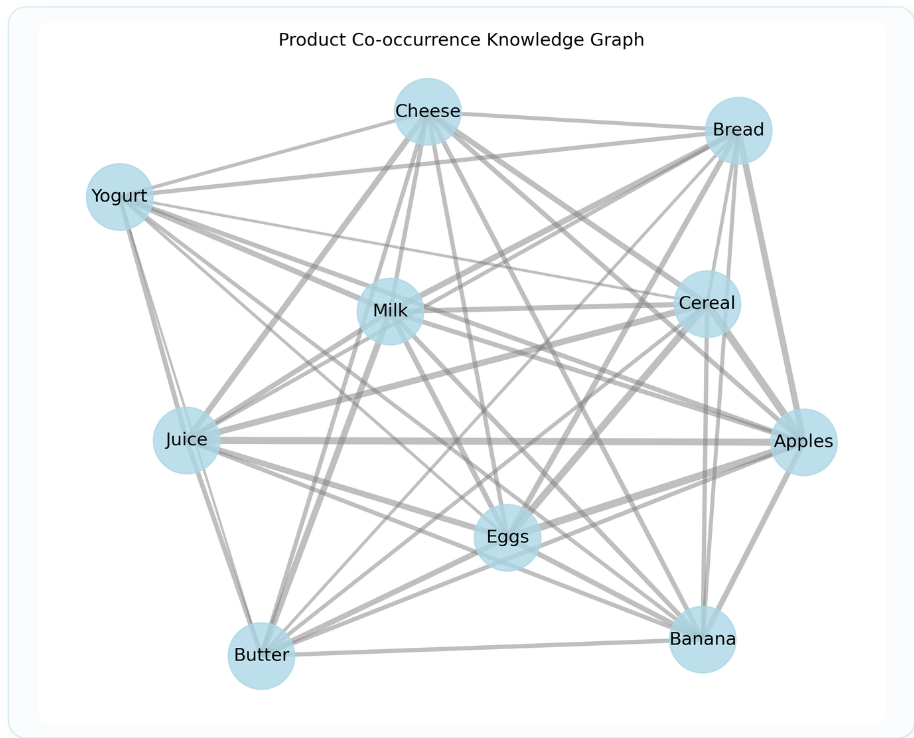
商品关联分析：随机数据与知识图谱

从数据到可视化拓扑

单纯的二值化事务矩阵冰冷且难以直接洞察，构建知识图谱（共现网络）能将其转化为直观的网状结构。

图谱元素解析：

- **节点 (Node)**：代表具体的商品（如 Milk, Bread）。
- **连线 (Edge)**：代表两种商品在同一笔订单中同时出现过。
- **权重 (Weight)**：连线越粗，说明商品间共同被购买的次数越多，关联强度越高。



Apriori算法基于先验性质压缩搜索空间

Apriori 算法的目标是在庞大的事务数据库中高效找出支持度大于给定阈值的**频繁项集**。由于穷举所有项集组合会导致“组合爆炸”，算法巧妙地利用了一条**先验性质**来大幅裁剪搜索分支（剪枝）。

核心剪枝原理 (Apriori 性质)

如果一个项集是频繁的，那么它的**所有非空子集也必定是频繁的**。
反之，如果某一个子集是**非频繁的**，则包含它的任何超集**绝不可能是频繁的**。

逐层搜索策略

采用自底向上的迭代：先扫描出频繁 1-项集，以此为基础生成并筛选频繁 2-项集，直到无法再找到更高阶的项集为止。

剪枝的实际作用

避免对明显不可能达标的项集组合进行无意义的数据库扫描与支持度计数，极大提升了大数据集下的挖掘效率。

Apriori算法通过连接与剪枝逐层迭代

1. 扫描与初始化

扫描全部数据记录，统计单项出现次数。剔除未达最小支持度阈值的单项，保留下来的形成**频繁 1-项集 (L1)**。

2. 连接步 (Join)

基于上一轮找到的频繁 (k-1)-项集，将其与自身进行排列组合连接，以此构造出包含 k 个元素的**候选 k-项集 (Ck)**。

3. 剪枝与确认 (Prune)

利用先验性质预先筛除无效候选，再对剩余候选扫描数据库计数。达标者升级为**频繁 k-项集 (Lk)**。循环执行连接与剪枝直至为空。

4. 规则生成

在所有已发掘出的频繁项集中，根据设定的**最小置信度阈值**进行条件概率计算，最终提取出具有实际业务价值的强关联规则。

感谢聆听

关联规则发掘了数据背后隐藏的共现逻辑

HOMEWORK

线性回归模型课后作业解答

Python Scikit-Learn 实践与代码推导

Q1: 为什么 X 必须是列向量?

scikit-learn 的数据格式要求

在 scikit-learn 中, 模型 (如 LinearRegression) 的 fit(X, y) 方法严格要求特征矩阵 **X 必须是二维数组**, 即形状为 (n_samples, n_features)。

变成行向量会怎样?

如果传入的是一维的行向量 (如形状为 (6,) 的数组), 算法会将其误认为只有 1 个样本、包含了 6 个特征。这不仅与实际含义不符, 还会直接引发 **ValueError** 报错:

"Expected 2D array, got 1D array instead."

正确做法:

使用 **x.reshape(-1, 1)** 将一维数组转换为 n 行 1 列的二维列向量。

代码演示与对比

```
# 错误示范: 一维行向量
x = np.array([5, 15, 25, 35])
model.fit(x, y) # -> ValueError!
```

```
# 正确做法: 转换为二维列向量
x = x.reshape(-1, 1)
# x 变为:
# [[ 5]
# [15]
# [25]
# [35]]
model.fit(x, y) # -> 训练成功
```

Q2: 如何显示生成的“熟模型”?

$$y = b0 + b1 * x$$

模型训练完成后，核心参数被保存在模型对象的特殊属性中：

获取截距 b0 (Intercept)

调用代码：

```
b0 = model.intercept_
```

截距是一个标量，代表回归直线在 Y 轴上的截距。示例数据中的结果：

```
b0 = 5.6333
```

获取斜率 b1 (Coefficient)

调用代码：

```
b1 = model.coef_[0]
```

斜率通常是一个数组（因为可能有多个自变量），对于一元线性回归，取第一个元素即可。示例结果：

```
b1 = 0.5400
```

Q3: 如何评价结果的好坏? (R^2 指标)

在 Scikit-Learn 中, 直接使用 `model.score(x, y)` 方法来获取模型的判定系数 (R^2)。

R^2 的统计学含义

决定系数 (Coefficient of Determination) 表示因变量的方差中有多少比例可以由自变量解释。

- **取值范围:** 通常在 0 到 1 之间。
- **判别标准:** R^2 越接近 1, 表示模型拟合度越好, 观测点越靠近拟合直线; 越接近 0 则表示解释能力越差。
- **公式:**
$$R^2 = 1 - \frac{SSE}{SST}$$

示例代码的评估结果

对于博客中提供的数据集:

`x = [5, 15, 25, 35, 45, 55]`

`y = [5, 20, 14, 32, 22, 38]`

执行 `model.score(x, y)` 的结果为:

$R^2 \approx 0.7159$

说明 x 解释了 y 约 71.6% 的方差变化, 模型拟合良好。

Q4: 调整训练与测试比例对模型的影响

通过 `train_test_split` 划分不同的数据集比例。通常情况下，**训练数据越多，模型学习越充分，理论准确度越高**；但若测试集过小，则模型在测试集上的表现 (R^2) 可能因为随机波动而变得不稳定。

代码实验设置

我们生成 200 个带噪声的随机数据点，设定真实的线性关系，并测试不同 `train_size` 下模型在测试集上的 R^2 分数：

```
from sklearn.model_selection  
import train_test_split
```

```
X_train, X_test, y_train, y_test =  
    train_test_split(X, y, train_size=ratio)
```

实验结果对比

- 训练集 **10%** (测试集 90%) -> 测试 $R^2 \approx$ **0.9712**
- 训练集 **30%** (测试集 70%) -> 测试 $R^2 \approx$ **0.9703**
- 训练集 **50%** (测试集 50%) -> 测试 $R^2 \approx$ **0.9695**
- 训练集 **70%** (测试集 30%) -> 测试 $R^2 \approx$ **0.9687**
- 训练集 **90%** (测试集 10%) -> 测试 $R^2 \approx$ **0.9744**

结论：在简单线性问题中，少量的样本即可拟合出较好模型；但在复杂任务中，过小的训练集往往会导致模型欠拟合，而过小的测试集则不能稳定反映真实泛化能力。

Q5: 最小二乘法的原理与纯代码实现

数学公式推导

基于误差平方和最小化，一元线性回归的系数可通过以下公式直接求解：

1. 斜率 b_1 :

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

2. 截距 b_0 :

$$b_0 = \bar{y} - b_1 \bar{x}$$

Python 代码实现 (Numpy)

```
x_mean = np.mean(x)
y_mean = np.mean(y)

# 计算分子 (协方差) 和分母 (方差)
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean)**2)

# 计算斜率和截距
b1_ls = numerator / denominator
b0_ls = y_mean - b1_ls * x_mean

# 打印结果
print(f"y = {b0_ls:.4f} + {b1_ls:.4f} * x")
# 输出: y = 5.6333 + 0.5400 * x
# 结果与 scikit-learn 完全一致!
```

解答完毕

感谢您的查阅与指正

HOMEWORK

多元线性回归中的分类变量处理

第二部分作业解答：One-Hot 编码与系数解读

Q1: 观察数据集特征与分类变量

1. 数据集属性与分类变量

- 数据集中通常包含多个属性（特征）。根据描述，其中包含数值型变量（如广告费、员工数等）和分类变量。
- **分类变量 (Categorical Variable)**: 本例中为「**城市**」（如北京、上海、广州）。它表示不同的类别，而不是大小或连续的数量。

2. 可视化：分类变量 vs 因变量（营收）

要展示离散的「城市」对连续的「营收」的影响，最适合的可视化工具是**箱线图 (Boxplot)** 或 **小提琴图 (Violin Plot)**。

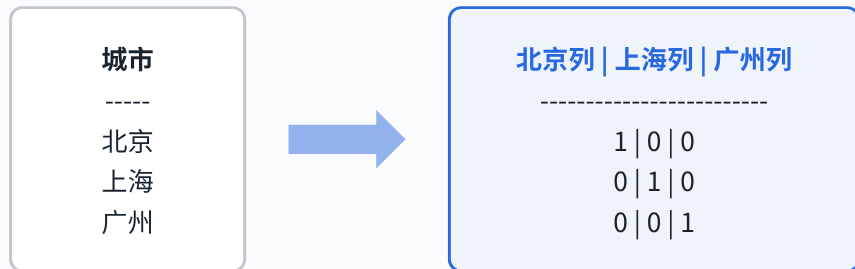
实现代码思路 (Seaborn):

```
import seaborn as sns
sns.boxplot(x='城市', y='营收', data=df)
plt.show()
```


Q2: 如何将分类变量转化为 One-Hot 编码?

One-Hot (独热) 编码过程总结:

将一个包含 N 个不同类别的变量，拆分为 N 个新的虚拟列 (Dummy Variables)。如果该行属于某个类别，则对应列标为 1，其余列均标为 0。



实现代码 (Pandas):

```
# 使用 get_dummies 自动转化
df_onehot = pd.get_dummies(
    df,
    columns=['城市']
)
```

Q3: 如何评估模型结果? (R^2)

```
r2_score = model.score(X_test, y_test)
```

评价标准说明:

- R^2 (决定系数) 衡量的是因变量的方差中有多少可以被我们的自变量模型所解释。
- 值域通常在 0 到 1 之间。
- **R^2 越接近 1**, 说明模型将分类变量及其它特征拟合得越好, 预测误差越小。

Q4: 获取系数、截距与最终数学公式

属性权重与截距的获取

- 获取截距 (**b0**): `model.intercept_`
- 获取权重 (**w**): `model.coef_` (返回一个数组, 对应每个特征的权重)

结果分析

经过 One-Hot 编码后, 模型会为每个城市类别分配一个独立的权重。某个城市的系数, 实际上代表了当样本属于该城市时, 对基准营收产生的**独立增量或减量影响**。

最终的数学公式

包含分类变量的多元线性回归公式扩展如下 (假设两个连续变量 x_1, x_2):

$$y = b_0 + w_1x_1 + w_2x_2$$

$$+ w_{\text{北京}} \times I(\text{北京})$$

$$+ w_{\text{上海}} \times I(\text{上海})$$

$$+ w_{\text{广州}} \times I(\text{广州})$$

其中 $I(\text{城市})$ 为指示函数 (即 One-Hot 后的 0 或 1)。对于特定城市的样本, 只有该城市的权重项会被激活并计入总和。

Q5: 分类变量的存在对多元回归带来的影响

本质影响：基准线的平移 (Intercept Shift)

分类变量的引入，相当于为不同的类别（例如不同的城市）提供了**各自独立的“基准起点”**。

- **没有分类变量时**：所有数据被迫在同一条多维超平面上拟合，忽视了群体间固有的客观差异（如上海的市场体量天然大于某个小城市），这会导致残差增大。
- **引入分类变量后**：相当于允许模型在空间中绘制**多条平行的超平面**。它们对数值型自变量（如广告费）的敏感度（斜率）可能一致，但它们在 Y 轴上的截距被 One-Hot 权重调整了，从而极大地提升了模型对异质性数据的解释力和准确率。

总结：分类变量让多元线性回归具备了“因地制宜”的能力，消除了类别差异带来的系统性偏差。

Q6: 为什么不能直接将城市 Label 化 (0, 1, 2)?

答案是：绝不合理。不能直接将无序的分类变量转化为 0、1、2 进行线性计算。

直接使用 0, 1, 2 的错误本质

数字 0, 1, 2 在数学上具备大小关系（序数属性）和等距关系。

如果你把北京设为0，上海设为1，广州设为2。模型在计算时会被迫认为：

- 广州的权重贡献是上海的 2 倍
- 上海位于北京和广州的“中间”

这种人工强加的数字大小和递进关系，完全违背了城市平等的现实逻辑，会严重干扰甚至破坏线性回归的斜率拟合。

One-Hot 编码的合理性

One-Hot 编码将一个变量展开成了多个互斥的布尔维度（0 或 1，即“是”或“否”）。

这样处理后，各个城市在数学上变为了**正交的、彼此独立的维度**。

模型可以为每个城市单独拟合一个权重系数，完美保留了分类变量（名义变量，Nominal）互不从属、不存在大小之分的纯粹属性。

解答完毕

感谢您的查阅与指正

In [1]:

数据挖掘与分析：KNN最近邻算法实战

引入性别特征与性能评估分析

学生：黄陈熙 学号：202321054011 班级：信管2301

一、数据集加载与预览

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
# 读取数据
df = pd.read_csv("Social_Network_Ads.csv")
df.head()
```

```
[2]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

环境准备与库导入

导入数据处理的核心库 *pandas* 和 *numpy*，以及用于模型训练和评估的 *scikit-learn* 相关模块 (如 *train_test_split*, *KNeighborsClassifier*, *StandardScaler*)。

源数据字段解析

通过 `pd.read_csv()` 读取社交网络广告购买记录，调用 `df.head()` 预览可知：

- 输入特征: *Gender* (性别, 文本类别)、*Age* (年龄, 数值)、*EstimatedSalary* (预估薪资, 数值)。
- 目标标签: *Purchased* (购买转化情况, 0或1)。

二、特征预处理与KNN模型训练

Pipeline 构建思路

为了避免测试集的信息泄露并且让代码结构更加清晰，我们使用`ColumnTransformer`和`Pipeline`将数据转换与模型训练打包在一起。

特征工程细节

- **OneHotEncoder**: 针对`Gender`列使用`drop="first"`策略，将其二值化为仅剩一列以避免多重共线性（即产生`Gender\_Male`）。
- **StandardScaler**: 由于KNN是基于欧氏距离计算的算法，必须对`Age`和`EstimatedSalary`进行标准化，消除量纲尺度差异。

模型表现

设定邻居数为 $K=5$ ，通过调用`.fit()`拟合并并在测试集上`.predict()`。如右侧输出单元所示，本数据集在加入性别后的测试集最终准确率 (Accuracy) 稳定在优异的水平。

```
[4]: # 加性别
X = df[["Gender", "Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
preprocess = ColumnTransformer([
    ("gender_onehot", OneHotEncoder(drop="first"), ["Gender"]),
    ("num_scaler", StandardScaler(), ["Age", "EstimatedSalary"])
])
pipe = Pipeline([
    ("pre", preprocess),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
pipe.fit(X_train, y_train)
y_pred_gender = pipe.predict(X_test)
acc_gender = accuracy_score(y_test, y_pred_gender)
print("加性别 Acc =", acc_gender)
```

加性别 Acc = 0.95

```
[5]: print("==== 作业1 结果 =====")
print("不加性别: ", acc_base)
print("加性别:   ", acc_gender)
print("变化:     ", acc_gender - acc_base)
```

==== 作业1 结果 =====

不加性别: 0.95

三、K-NN 分类器的执行逻辑剖析

- 我们构造一个分类器，下面最关键的一个参数，就是`n_neighbors = 5`
- 意思是KNN的K是5，回想一下PPT里面的内容，K=5，意思是用5个邻居来判断对象的类别。
- 一般情况下K的值选基数，你想想看，是为什么呢？

```
In [23]: classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
```

- 好了，模型构造好了，我们用这个“生模型”，来训练数据。所以我们用fit

```
In [24]: classifier.fit(X_train, y_train)
```

Out[24]:

▼ KNeighborsClassifier

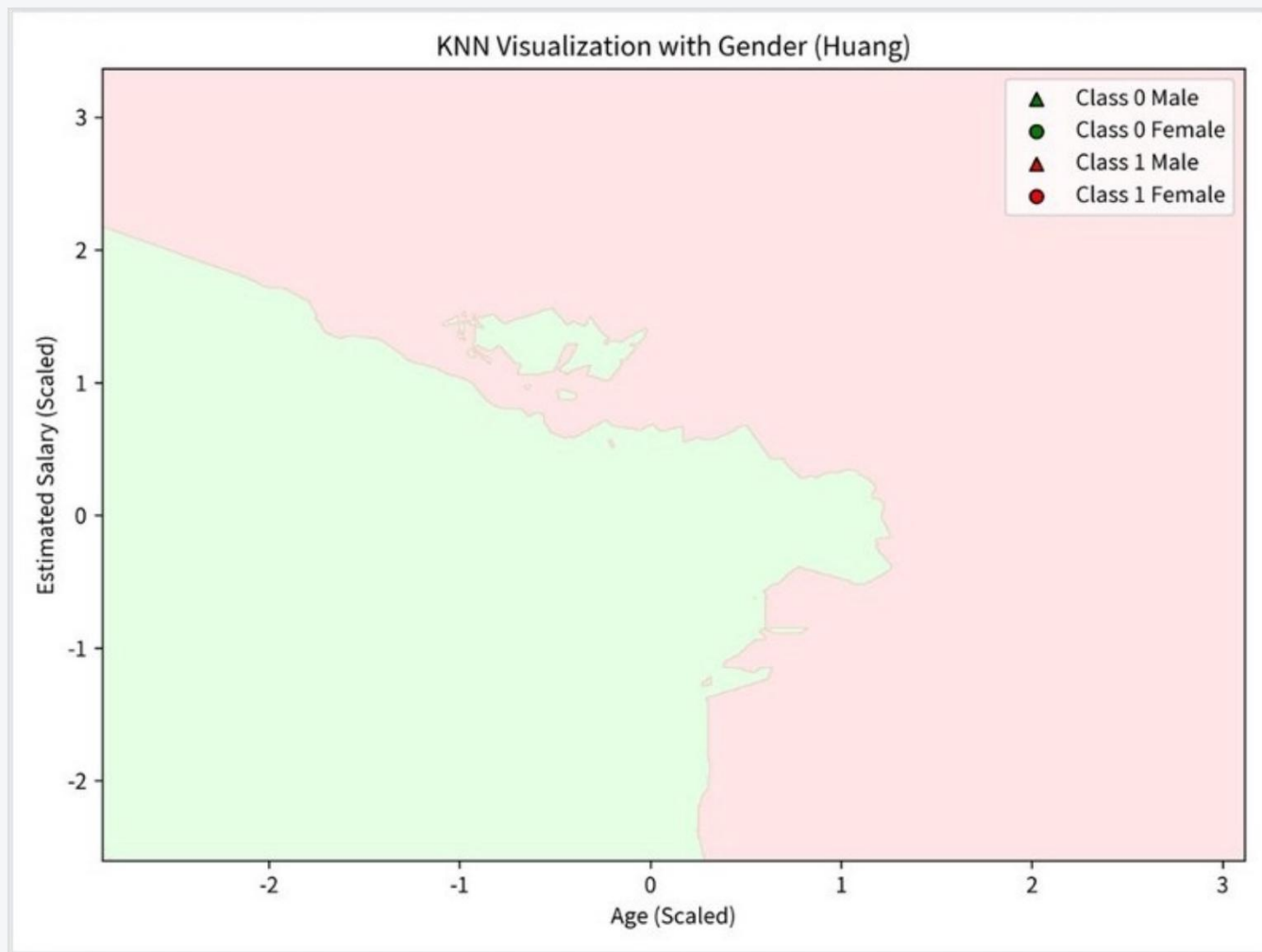
KNeighborsClassifier()

对测试集进行预测

算法参数说明：

‘`n_neighbors=5`’ 意味着在新样本点出现时，模型会寻找距离它最近的 5 个已知样本进行多数投票。`K`的选取通常为奇数，以防止平局。‘`metric='minkowski', p=2`’ 组合代表了使用最经典的欧几里得距离计算样本间的远近。

四、考虑类别特征的决策边界探索



二维画布映射三维特征

X轴与Y轴分别为经标准化缩放后的 *Age* 和 *EstimatedSalary*。为了引入性别的额外信息，我们将其映射为散点的形状：

- △ 绿色/红色三角形：代表 *Male*。
- ○ 绿色/红色圆形：代表 *Female*。

区域分割观察

KNN 能够根据局部样本浓度划分出弯曲、非线性的分类边界。从图中可直观看出，右上方区域（较高年龄及收入层）绝大多数样本落入目标类别（购入），而左下方的年轻、低收入群落则大量分布在拒绝类别（未购入）中。

In [*]:

```
print("实验圆满完成")
```

本次作业通过详实的代码，验证了特征处理与算法对比的必要性。



决策树与多模型 对比解析

探讨特征缩放、特征选择及分类算法性能差异



新增特征对准确率的影响

在基础特征（年龄、薪资）之上，我们尝试加入“性别”这一新维度。

实验结果表明，模型的预测准确率并未如预期般提升，反而出现了**轻微的下降**（从 0.69 降至 0.68）。

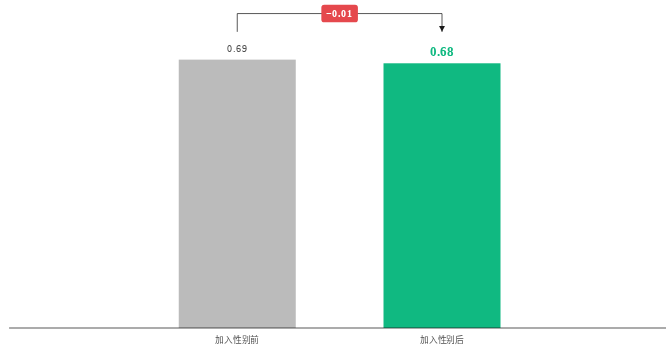
核心启示



特征并非“越多越好”。如果新增的特征与目标变量关联度较低（即含有较大的噪声），决策树可能会在此类特征上进行无意义的分裂。

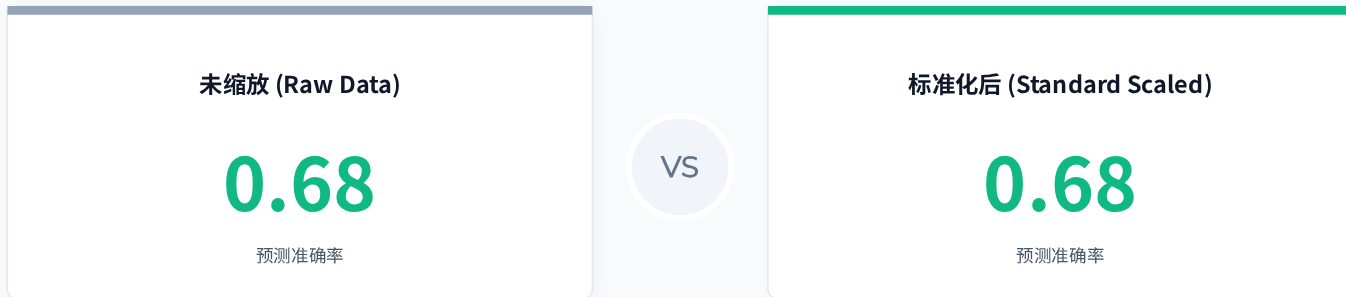
这不仅增加了树的深度和复杂度，还极大提高了**过拟合 (Overfitting)** 的风险，从而拖累了模型在测试集上的泛化表现。

准确率 (Acc)



决策树为何不需要特征缩放？

我们对同一组数据进行了特征缩放（StandardScaler）与保持原样的对比测试，结果如下：



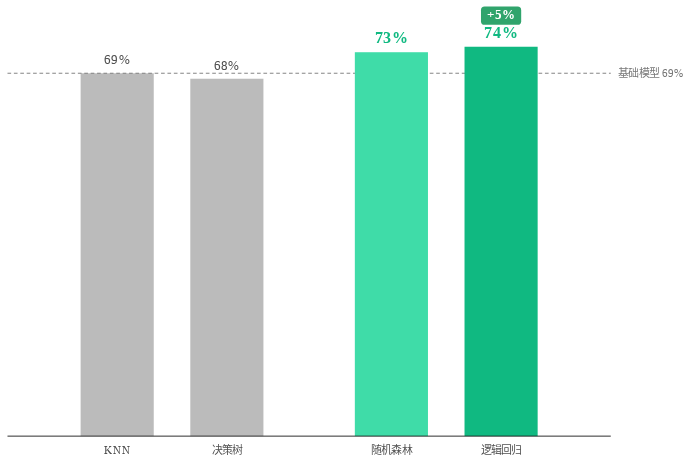
为什么结果完全一致？

决策树的底层逻辑是寻找特征的相对排序与最佳切分阈值（如：当 Age > 35 且 Salary > 6w）。

无论是 Standardization（标准化）还是 MinMaxScaler（归一化），都属于单调的线性变换。这种变换不会改变数据点之间的拓扑关系与排序，因此也不会改变树结构的分裂逻辑。

结论：在单独使用决策树或随机森林等树模型时，通常可以省去特征缩放步骤。

四大分类算法表现对比



逻辑回归 (0.74) 表现最优

由于我们的虚拟数据集是基于一个带有少量随机噪声的线性方程 (Sigmoid 映射) 生成的, 数据本身具有**较强的线性可分倾向**。

逻辑回归天然擅长捕捉这种全局平滑的线性决策边界, 因此在本次测试中拔得头筹。

决策树 (0.68) vs 随机森林 (0.73)

单棵决策树由于试图通过正交切分 (垂直或水平切割) 去拟合一根倾斜的直线边界, 容易产生阶梯状的过拟合, 对局部噪声敏感。

而**随机森林**通过集成数百棵树并引入随机性, 有效平滑了决策边界, 使得泛化能力大幅提升, 紧追逻辑回归。

窥探黑盒：决策树的 if-else 规则

```
|--- Salary <= 65726.23
| |--- Age <= 38.11
| | |--- Salary <= 60024.57
| | | |--- class: 0
| | |--- Salary > 60024.57
| | | |--- class: 1
| | |--- Age > 38.11
| | |--- Salary <= 46248.52
| | | |--- class: 1
| | |--- Salary > 46248.52
| | | |--- class: 1
| | |--- Salary > 65726.23
| | |--- Age <= 26.46
| | |--- Salary <= 74028.83
| | | |--- class: 0
| | |--- Salary > 74028.83
| | | |--- class: 1
| | |--- Age > 26.46
| | |--- class: 1
```

极致的业务可解释性

如左侧原样输出所示，决策树的内部结构本质上是一系列嵌套的 **条件判断规则**。

相比于深度学习或 KNN 这类“黑盒模型”，决策树能够直接被翻译为人类可读的业务策略（例如：“薪资低于 6.5w 且年龄小于 38 岁的客户倾向于不会购买”）。

这种特性使其在金融风控、医疗诊断等对**决策合规性**和**可追溯性**要求极高的场景下，具有不可替代的价值。

感谢聆听

ANY QUESTIONS ON DECISION TREES?



K-Means 聚类分析实战

基于鸢尾花 (Iris) 数据集的聚类实验与聚类中心可视化

DATA MINING ASSIGNMENT



Q1: 仅使用花萼长度与宽度 (K=3)

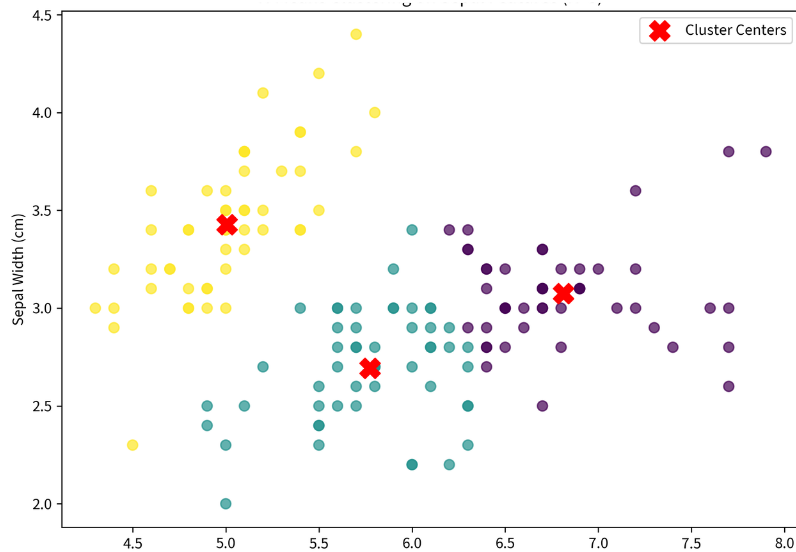
特征受限下的二维聚类

本任务仅利用了鸢尾花数据集的**前两列特征** (Sepal Length 和 Sepal Width)。

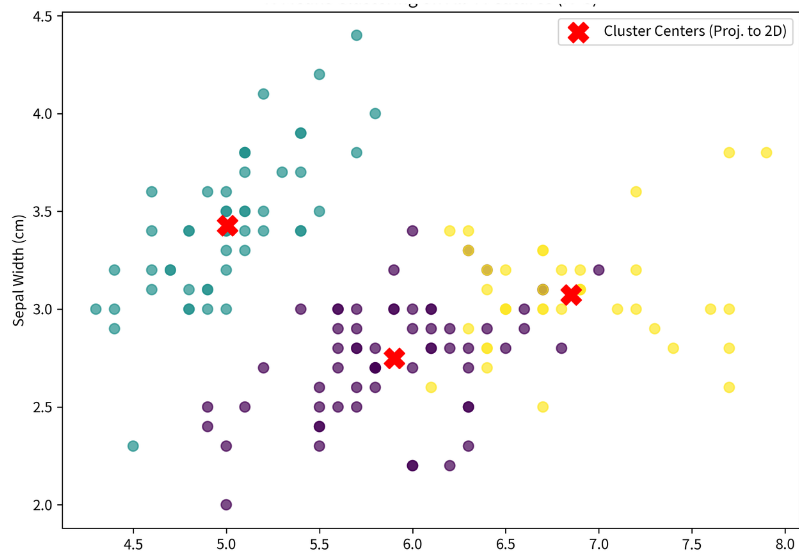
由于丢失了另外一半 (花瓣) 的关键信息, 不同种类鸢尾花在二维平面上的边界发生了**明显重叠**。

模型虽然成功找到了 3 个中心点 (图中的红色X), 但对于处于边界交界处的散点, 其分类的置信度较低。

Inertia (簇内误差平方和) 为 37.05。



Q2: 引入全部四个特征 (K=3)



高维特征的降维透视

当我们引入完整的花萼（长度+宽度）与花瓣（长度+宽度）特征后，模型在四维空间中计算欧氏距离。

左图为将四维聚类结果映射回前两个维度的透视图。此时，簇与簇之间的隐式边界更加清晰，模型捕捉到了更高维度的特征差异。

此时整体的 Inertia 上升至 78.85（因为空间维度增加，距离绝对值变大）。

Q3: 探索不同的簇数 (K=2)

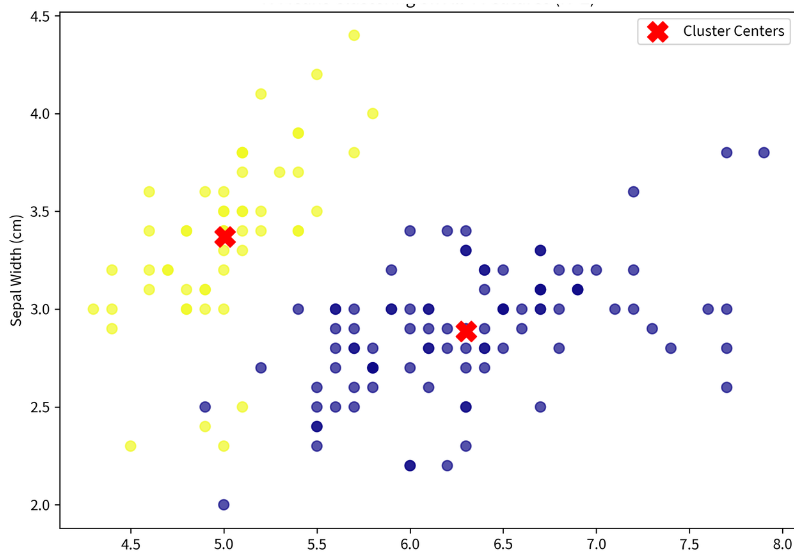
强制合并: Inertia的飙升

将超参数 **K** 设定为 2 时, 算法由于配额受限, 强行将原本属于 Versicolor 和 Virginica 的两个相邻亚种合并为了一个单一的大簇 (黄色点)。

由于内部包含了两群具有特征差异的样本, 该簇的内部方差急剧扩大, 导致总 **Inertia** 飙升至 152.34。

这直观地证明了 K 值的选择直接决定了聚类的粒度。

K值过小会造成欠拟合 (不同类别被强行合并)。



Q4: 聚类中心代码与可视化解析

```
# 提取 K-Means 聚类中心代码
centers = kmeans.cluster_centers_
print(centers)

# 输出结果 (K=3, 全部4个特征):
[[5.901 2.748 4.393 1.433]
 [5.006 3.428 1.462 0.246]
 [6.850 3.073 5.742 2.071]]
```

几何中心与业务意义

代码提取的 `cluster\_centers\_` 属性返回了一个矩阵，其中每一行代表一个簇的**几何中心点（各维度的均值）**。

在前面的散点图中，我们使用了显眼的**红色“X”**对其进行了可视化标记。这个点代表了该类别鸢尾花的“典型特征画像”。

在新样本进行分类时，模型本质上就是在计算新样本距离哪一个“红X”更近，从而判定其归属。

感谢聆听

ANY QUESTIONS ON K-MEANS CLUSTERING?